Technical University of Denmark



34749 Playware technology - programming app control of playful embedded system

Manual For Motocube

Contents

1 Introduction **1**
1.1 Overall Structure of Setup 1

2 Overview of Hardware Used **1**
2.1 Bluetooth Low Energy Module 2
2.2 Node MCU ESP32 4
2.3 ESP32 BLE Services 4

3 Software **5**
3.1 About App 5
3.2 ESP32 BLE Implementation 6

4 Softwares Required **7**
4.1 Ionic Cli 7
4.2 Visual Studio Code 7
4.3 Arduino IDE 8
4.4 Android Studio IDE 8

5 Steps to Run App **8**
5.1 About Code 8

6 Steps to Setup ESP32 **8**
6.1 Libraries 9
6.2 About Code 9

7 Next Steps **9**

8 Code **10**

References **15**

1 Introduction

In this document, the reader will get to know how to setup the MotoCube and the BLE modules provided in the course and a short insight of the Art app.

1.1 Overall Structure of Setup

In the project, we focus on controlling actuators with help of BLE modules. By interfacing the BLE modules with a preexisting art app we get the data from the modules to the app and from that, we use Bluetooth to send the data from the app to a NodeMCU ESP32, which receives the data and controls the output actuators.

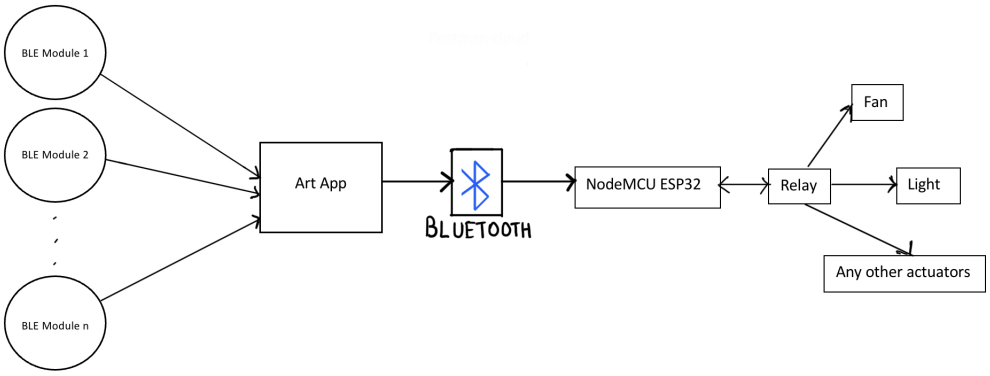


Figure 1: Block diagram of the system

Figure 1 shows the overall architecture of the system with the parts and connections. With the orientations of the BLE modules, we can control the state of the actuators. The user can add extra actuators as per their requirement. These can be set up as the user wants by selecting the options in the app. The modules can also be selected to be the controller of the specific actuators based on the user's preference. With the use of Bluetooth, we get faster data transmission over smaller ranges in the same room.

2 Overview of Hardware Used

The NodeMCU ESP32 plays the part of actuating the actuators connected to the NodeMCU by setting the state in the output pins. Figure 2 shows a small circuit connection of the NodeMCU with a multicolor LED, where the node MCU receives the data via Bluetooth and controls the LED state and color based on the BLE module input.

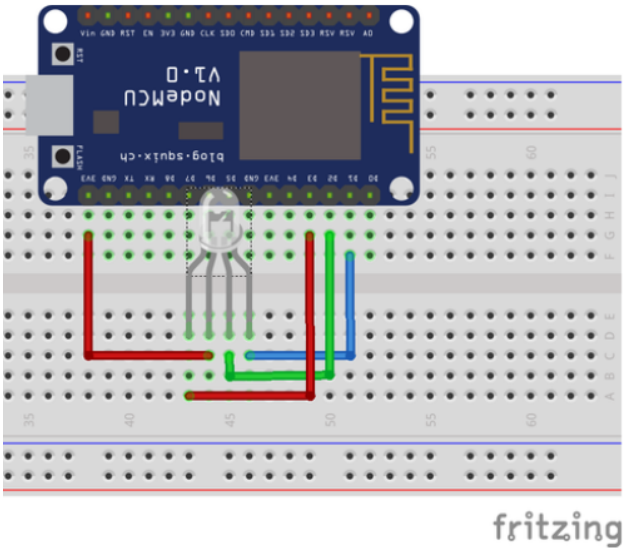


Figure 2: ESP32 connected to RGB Led

2.1 Bluetooth Low Energy Module

Bluetooth low energy is a wireless personal area network technology. It is a low power consuming Bluetooth device that can be used without changing of battery for months or years. It can be used in various applications like smart home, health, sport and fitness to name a few. Like classic Bluetooth, it is also a wireless communication standard working on the 2.4 GHz band focusing on ultra-low power applications in short ranges.
The modules used uses Bluetooth 5.0 communication protocol for sending and receiving of data.

Figure fig.3 shows the architecture of Bluetooth Low Energy. With ultra-low power and a combination of turning the radio on and off as soon as possible, the coin cell battery can last longer than a year. Bluetooth Low Energy features small communication packets, a maximum of 244 Bytes, and fast connection and disconnection times, lower than six milliseconds[1].

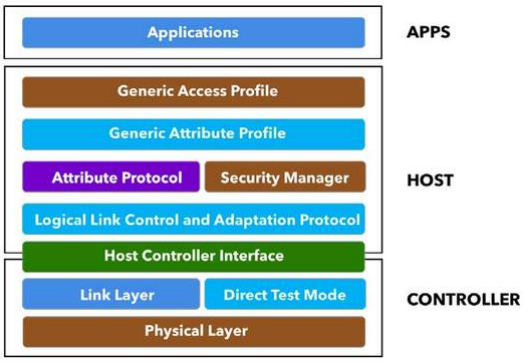


Figure 3: Bluetooth Low Energy Architecture

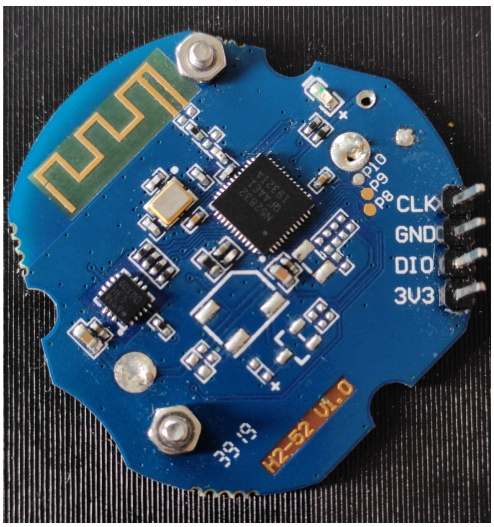


Figure 4: Bluetooth Low Energy Module Hardware

nRF52832 Systemonchip (SoC)

The Above figure fig.4 shows the Bluetooth Low Energy Module hardware. The big chip in the module is the nRF52832 SoC by Nordic Semiconductor. The nRF52 series offers options for Bluetooth Low Energy, proprietary 2.4 GHz, and ANT giving freedom for the user to focus on the development of their wireless system. The peak current for RX/TX is only 5.5mA and allows data rates of 2Mbps.

LIS3DH MEMS Accelerometer

The smaller chip on the left is the LIS3DH MEMS accelerometer. It features low-power modes that allow advanced power saving and smart embedded functions. Depending on the operating mode it consumes 2-11 μ A.
It consists of an integrated FIFO buffer that allows storing of data and also capable of generating interrupt signals for different events caused by the position change of the device. Apart from this, the device has inbuilt functions to detect 6D and 4D orientation, freefall detection, and motion detection. The below figure fig.5 shows the orientations that can be used. We use the 6 orientations to represent different states in the app.

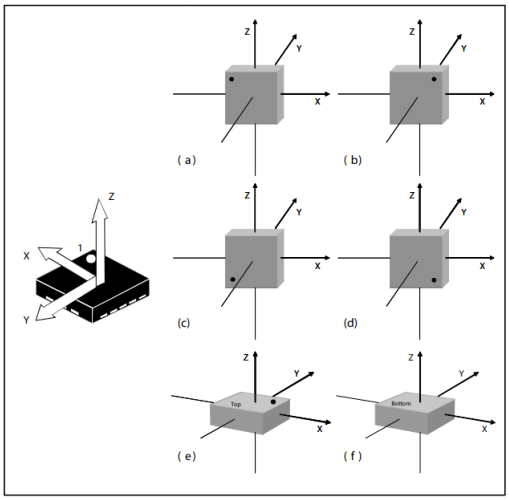


Figure 5: 6D Recognised positions

2.2 Node MCU ESP32

The NodeMCU ESP32 module is a compact prototyping and IoT development board, consisting of a 240 MHz 32 bit RISC processor. It has a 2.4 GHz dual-mode WiFi and a BT wireless connection. Moreover, the microcontroller has integrated: a 512 kB SRAM and 4 MB memory, 2x DAC, 15x ADC, 1x SPI, 1x I ² C, 2x UART. PWM is activated at all digital pins. We program the NodeMCU to actuate the output actuators based on the input it receives.

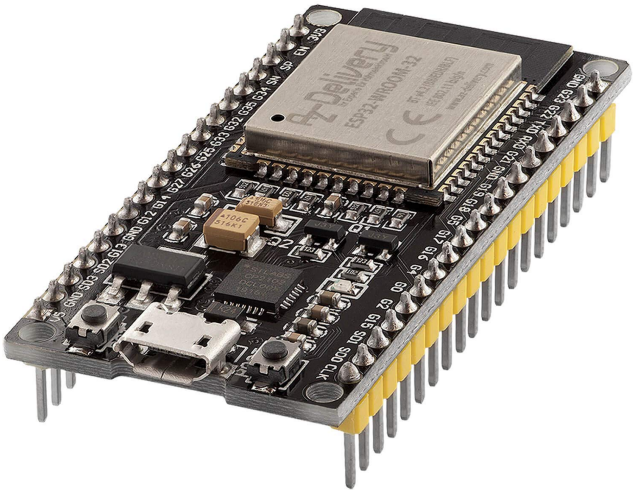


Figure 6: ESP32 with CP2102 Chip

The ESP32 has 2.4 GHz dual-mode Wi-Fi and Bluetooth chips from TSMC as well as 40 nm low-power technology.
The WLAN function in the ESP32 supports all common encryption methods such as WPA2. It can also act as an access point or sniffer in passive mode in the WLAN.

2.3 ESP32 BLE Services

As mentioned in the previous section, we have both Bluetooth and WiFi ESP32. The Bluetooth 4.0 specification is the integration of BLE. This is a highly optimized version of Classic Bluetooth designed specifically for low-power wireless communication. It was created for short-distance communication with low power consumption and designed with IoT and battery-operated applications in mind. From figure.3 we can see the Architecture of the BLE.

BLE States

A BLE device can have 5 possible states:

- Standby
- Advertising
- Scanning
- Initiating

- Connected

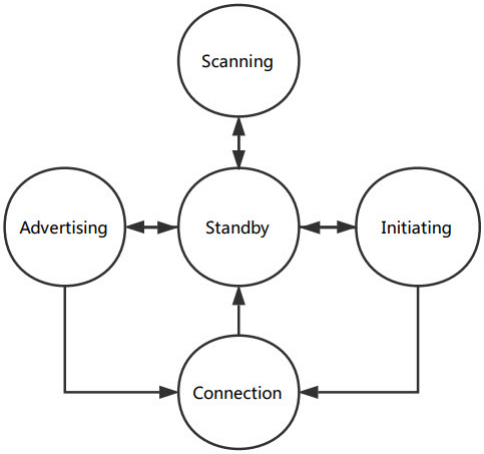


Figure 7: BLE State Diagram

ESP32 BLE Server Client Model

Any BLE device can be configured as a server or a client. Similarly, the ESP32 can also be configured as a Server or Client.

If configured as a Server, the ESP32 advertises its presence so the clients can read its data. Otherwise, if it's configured as a Client, the ESP32 scans for servers and makes a connection to receive data from the server.

3 Software

3.1 About App

We added a new mode called the IO Mode in the existing art app, the IO mode controls different output devices when the BLE module is rotated. For instance, we interfaced RGB Led with the ESP32 and rotating the dice on alternate positions would turn ON/OFF the less (Sides 1/4 for Red Led, 2/5 for Green Led, 3/6 for Blue Led). There were 2 implementations done to communicate the app with the ESP32 for actuation. The first implementation was done using an HTTPClient which works with Wifi. An HTTPClient was set up in the app software which would send postman commands to the ESP32. Similarly, a server was set up in the ESP32 to receive the commands. The other implementation was done using BLE services on the Ionic framework to interface the ESP32 with the App using BLE.

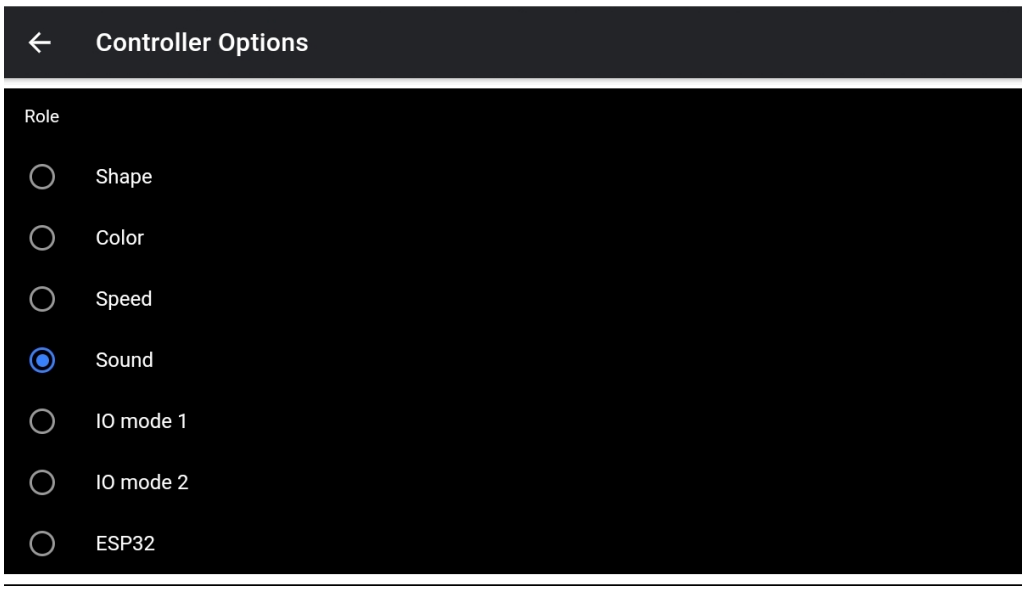


Figure 8: New ESP32 and I/O Mode Option

3.2 ESP32 BLE Implementation

We are using the BLE one-way communication as we only using the App to send data to the ESP32 NodeMCU to actuate the light connected to it.
The ESP32 acts as a Client that receives data after connecting to the App which acts as a Server.

Figure.9 shows the detection of the BLE module and also the NodeMCU(MotoCube) in the app selected it as I/O Mode and ESP32 respectively, where the BLE Module acts as the Input to the app and the ESP32 acts as output. The faces of the BLE module are sent as input to the MotoCube which on getting the side of the module, actuates the lights and fan based on the set conditions for the module face values(1-6) on the NodeMCU.

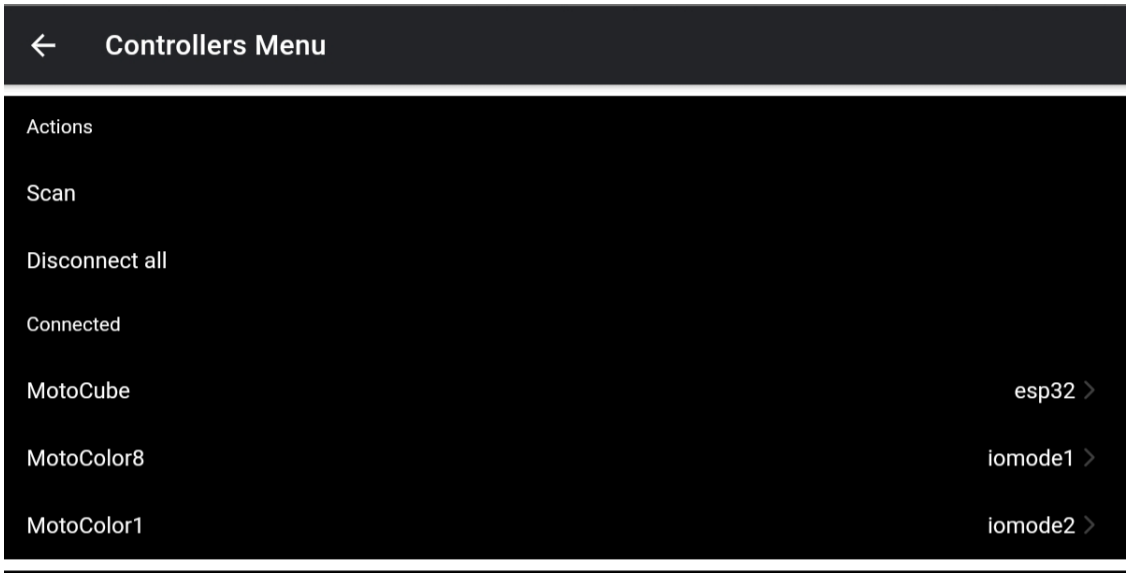


Figure 9: Connecting MotoCube(NodeMCU) and BLE Module to app on I/O Mode

The ESP32 acts as a client which communicates with the server (Paglia App) via BLE service. For

implementing the BLE service in the ESP32 we do it by creating 2 callback function: BLE Server Callback and BLE Characteristic Callback. The BLE Server Callback function has an onConnect (to detect if the ESP32 is connected) and onDisconnect (to detect if it got disconnected) function. The BLE Characteristic Callback has onWrite, onNotify, onRead, and onStatus functions to detect the active mode of the device. The data received from the app is stored in a variable and then the data is checked from the variable, If the data received is 1/4 - Turn On/Off Red Led, 2/5 - Turn On/Off Green Led, 3/6 - Turn On/Off Blue Led. The output from the serial monitor is shown in the fig10.

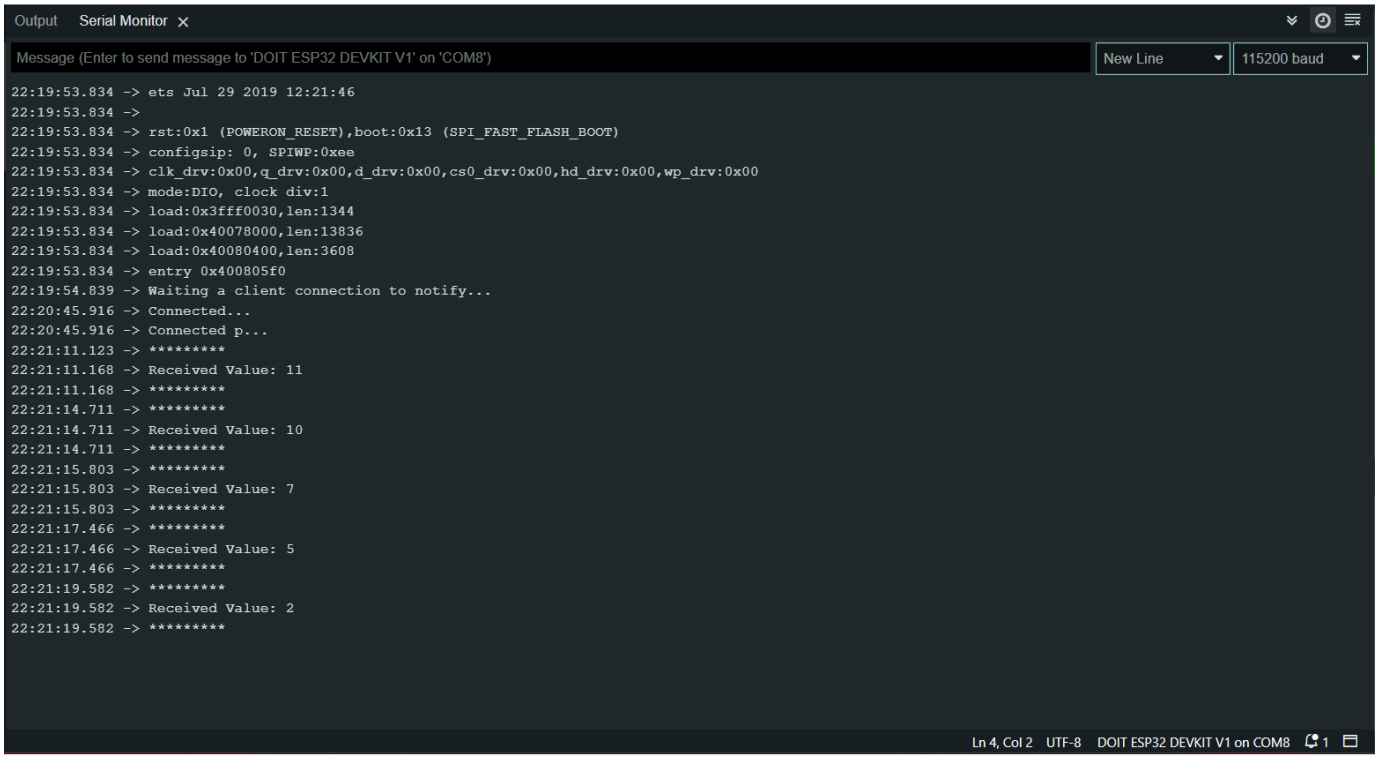


Figure 10: ESP32 Serial Monitor Output

4 Softwares Required

The use of Visual Studio Code is recommended as it is easier to use Softwares Installation required:

- Ionic Cli
- Visual Studio Code (For adding and editing app funtions)
- Arduino IDE (For ESP32 Code editing and uploading)
- Android Studio (To upload app to Tablet)

4.1 Ionic Cli

Follow the steps given on this site for installingIonic Cli.

4.2 Visual Studio Code

Link for installation of VS Code.

4.3 Arduino IDE

Link for installation of Arduino IDE.

4.4 Android Studio IDE

Link for installation of Android Studio

5 Steps to Run App

First we need to clone the repo from the github PagliaCode. To clone the repo either use Download zip from the code menu or use the cmd:

```
git clone https://github.com/MaxLeggieri/PagliaCode/tree/pjain/bleio
```

Mentioned below are the setup instruction for the art app:

```
cd App path to directory (PagliaCode directory)
ionic build --prod
ionic cap sync android
npx cap open android ~ Opens Android Studio
ionic build --prod && ionic cap sync android && npx cap open android.
npm run android:prod (Use this for full production execution)
npm run android:test (Use this for app test execution)
Run the Paglia code app
```

After the Ionic opens the Android Studio, the app can be loaded into the device or tablet in the form of apk from the android studio.

5.1 About Code

To create new modes, we have to first create a button in the ble controller options page this can be created by modifying the controller-options.page.html inside the controller-options folder. where ion-label is the label to be shown at the menu and iomode1 will be the role set function to be called later from the app.

```
<ion-item color="black">
    <ion-label>IO mode 1</ion-label>
    <ion-radio slot="start" value="iomode1"></ion-radio>
</ion-item>
```

Then after the button is created we will be adding a functionality to the mode which is done in the home.page.ts file. To add a mode functionality the code would look like

```
if (cube.role == "iomode1") {
  this.bleService.sendMessage(side)
}
```

6 Steps to Setup ESP32

Steps to execute a ble server on the ESP32 can be found on this tutorial - BLE ESP32 Tutorial

6.1 Libraries

Follow the steps mentioned here to add the ESP32 library to Arduino IDE. Also, install libraries necessary to for O/P devices connected to the ESP32. For example, in case you are using a LED strip or ring you need Adafruit_NeoPixel library

6.2 About Code

The ESP32 can be configured as a Server or a Client i.e., ESP32 can be either a Server, which advertises its presence so that clients can read its data or as a Client, which scans for servers and makes a connection to receive data from the server. The ESP32 acts as a client which communicates with the server (Paglia App) via BLE service. For implementing the BLE service in the ESP32 we do it by creating 2 callback functions: BLE Server Callback and BLE Characteristic Callback. The BLE Server Callback function has an onConnect (to detect if the ESP32 is connected) and onDisconnect (to detect if it got disconnected) function. The BLE Characteristic Callback has onWrite, onNotify, onRead, and onStatus functions to detect the active mode of the device. The data received from the app is stored in a variable and then the data is checked from the variable, If the data received is 1/4 - Turn On/Off Red Led, 2/5 - Turn On/Off Green Led, 3/6 - Turn On/Off Blue Led.

See the code section to see where to add your logic and edit the code as per your project requirement.

7 Next Steps

With the setup steps followed, the ArtApp and ESP32 are ready to use with the BLE modules.

Think of a logic to create a use for the app and ESP32 with the BLE modules to have a function as per your project idea.

8 Code

ESP32 Code

```
/*
    Video: https://www.youtube.com/watch?v=oCMOYS71NIU
    Based on Neil Kolban example for IDF:
        https://github.com/nkolban/esp32-snippets/blob/master/cpp_utils/tests/BLE%20Tests/SampleNotify.
    Ported to Arduino ESP32 by Evandro Copercini

   Create a BLE server that, once we receive a connection, will send periodic
       notifications.
   The service advertises itself as: 6E400001-B5A3-F393-E0A9-E50E24DCCA9E
   Has a characteristic of: 6E400002-B5A3-F393-E0A9-E50E24DCCA9E - used for receiving data
       with "WRITE"
   Has a characteristic of: 6E400003-B5A3-F393-E0A9-E50E24DCCA9E - used to send data with
       "NOTIFY"

   The design of creating the BLE server is:
   1. Create a BLE Server
   2. Create a BLE Service
   3. Create a BLE Characteristic on the Service
   4. Create a BLE Descriptor on the characteristic
   5. Start the service.
   6. Start advertising.

   In this example rxValue is the data received (only accessible inside that function).
   And txValue is the data to be sent, in this example just a byte incremented every
       second.
*/
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>
#include <Adafruit_NeoPixel.h>
#define PIN      21
#define PIN2     22
#define NUMPIXELS 12

Adafruit_NeoPixel pixels(NUMPIXELS, PIN, NEO_GRB + NEO_KHZ800);
Adafruit_NeoPixel pixels2(NUMPIXELS, PIN2, NEO_GRB + NEO_KHZ800);

BLEServer *pServer = NULL;
BLECharacteristic *pTxCharacteristic;
bool deviceConnected = false;
bool oldDeviceConnected = false;
uint8_t txValue = 0;

int red = 17;
int green = 18;
int blue = 19;

// See the following for generating UUIDs:
// https://www.uuidgenerator.net/
```

```
#define SERVICE_UUID "6e400001-b5a3-f393-e0a9-e50e24dcca9e"
#define CHARACTERISTIC_UUID_TX "6e400002-b5a3-f393-e0a9-e50e24dcca9e"
#define CHARACTERISTIC_UUID_RX "6e400003-b5a3-f393-e0a9-e50e24dcca9e"

// f2428fea-be8d-4c41-959f-745668400a4f

class MyServerCallbacks : public BLEServerCallbacks {
  void onConnect(BLEServer *pServer) {
    deviceConnected = true;
    Serial.println("Connected...");
  };

  void onDisconnect(BLEServer *pServer) {
    deviceConnected = false;
    Serial.println("Disconnected...");
  }

  void onConnect(BLEServer *pServer, esp_ble_gatts_cb_param_t *param) {
    deviceConnected = true;
    Serial.println("Connected p...");
  }
};

class MyCallbacks : public BLECharacteristicCallbacks {
  void onWrite(BLECharacteristic *pCharacteristic) {
    std::string rxValue = pCharacteristic->getValue();

    if (rxValue.length() > 0) {
      Serial.println("*********");
      Serial.print("Received Value: ");
      for (int i = 0; i < rxValue.length(); i++)
        Serial.print(rxValue[i]);

      Serial.println();
      Serial.println("*********");
    }
 // Send Message Button Test on App - To test connection on Serial Monitor
    if (rxValue == "Hi"){
        digitalWrite(red, HIGH); // turn the LED on (HIGH is the voltage level)
        delay(1000);                       // wait for a second
        digitalWrite(red, LOW);  // turn the LED off by making the voltage LOW
        delay(1000);                       // wait for a second

        digitalWrite(green, HIGH); // turn the LED on (HIGH is the voltage level)
        delay(1000);                       // wait for a second
        digitalWrite(green, LOW); // turn the LED off by making the voltage LOW
        delay(1000);

        digitalWrite(blue, HIGH); // turn the LED on (HIGH is the voltage level)
        delay(1000);                       // wait for a second
        digitalWrite(blue, LOW); // turn the LED off by making the voltage LOW
        delay(1000);
    }
 //Add Your logic here
    if (rxValue == "1"){
        for(int x=0; x<12; x++){
```

```
        pixels.setPixelColor(x, pixels.Color(70, 70, 0)); // Yellow
      }
      pixels.show();
      delay(2000);

  }

  if (rxValue == "2"){
      for(int x=0; x<12; x++){
        pixels.setPixelColor(x, pixels.Color(0, 70, 0)); // Green
      }
      pixels.show();
      delay(2000);

  }
  if (rxValue == "3"){
      for(int x=0; x<12; x++){
        pixels.setPixelColor(x, pixels.Color(0, 70, 70)); // Blue
      }
      pixels.show();
      delay(2000);

  }
  if (rxValue == "4"){
      for(int x=0; x<12; x++){
        pixels.setPixelColor(x, pixels.Color(70, 0, 0)); // Red
      }
      pixels.show();
      delay(2000);

  }
  if (rxValue == "5"){
      for(int x=0; x<12; x++){
        pixels.setPixelColor(x, pixels.Color(0, 0, 70)); // Dark Blue
      }
      pixels.show();
      delay(2000);

  }
  if (rxValue == "6"){
     for(int x=0; x<12; x++){
        pixels.setPixelColor(x, pixels.Color(70, 0, 70)); // Pink
      }
      pixels.show();
      delay(2000);
  }
pixels.clear();
  if (rxValue == "7"){
     for(int x=0; x<2; x++){
        pixels2.setPixelColor(x, pixels2.Color(70, 0, 70)); // Pink
      }
      pixels2.show();
      delay(1000);
      pixels2.clear();
  }
```

```
  if (rxValue == "8"){
     for(int x=2; x<4; x++){
       pixels2.setPixelColor(x, pixels2.Color(70, 70, 70)); // Pink
      }
      pixels2.show();
      delay(1000);
      pixels2.clear();
  }

  if (rxValue == "9"){

     for(int x=4; x<6; x++){
       pixels2.setPixelColor(x, pixels2.Color(70, 70, 0)); // Pink
      }
      pixels2.show();
      delay(1000);
      pixels2.clear();
  }

  if (rxValue == "10"){
     for(int x=6; x<8; x++){
       pixels2.setPixelColor(x, pixels2.Color(70, 0, 0)); // Pink
      }
      pixels2.show();
      delay(1000);
      pixels2.clear();
  }

  if (rxValue == "11"){
     for(int x=8; x<10; x++){
       pixels2.setPixelColor(x, pixels2.Color(0, 0, 70)); // Pink
      }
      pixels2.show();
      delay(1000);
      pixels2.clear();
  }

  if (rxValue == "12"){
     for(int x=10; x<12; x++){
       pixels2.setPixelColor(x, pixels2.Color(0, 70, 70)); // Pink
      }
      pixels2.show();
      delay(1000);
      pixels2.clear();
  }
  pixels2.clear();
 rxValue = "0";
 }
// The above lines are used as logic for LED Ring with 2 BLE Modules connected to app as
    seperate I/O Modes

 void onNotify(BLECharacteristic *pCharacteristic) {
  Serial.println("onNotify p...");
 }

 void onRead(BLECharacteristic *pCharacteristic) {
```

```
    Serial.println("onRead p...");
  }

  void onStatus(BLECharacteristic *pCharacteristic, Status s, uint32_t code) {
    Serial.println("onStatus p...");
  }
};


void setup() {
  Serial.begin(115200);
  pixels.begin();
  pinMode(red, OUTPUT);
  pinMode(green, OUTPUT);
  pinMode(blue, OUTPUT);
  // Create the BLE Device
  BLEDevice::init("MotoCube");

  // Create the BLE Server
  pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  // Create the BLE Service
  BLEService *pService = pServer->createService(SERVICE_UUID);

  // Create a BLE Characteristic
  pTxCharacteristic = pService->createCharacteristic(
    CHARACTERISTIC_UUID_TX,
    BLECharacteristic::PROPERTY_NOTIFY);

  pTxCharacteristic->addDescriptor(new BLE2902());

  BLECharacteristic *pRxCharacteristic = pService->createCharacteristic(
    CHARACTERISTIC_UUID_RX,
    BLECharacteristic::PROPERTY_WRITE);

  pRxCharacteristic->setCallbacks(new MyCallbacks());

  // Start the service
  pService->start();

  BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  pAdvertising->setScanResponse(true);
  pAdvertising->setMinPreferred(0x06);
  pAdvertising->setMinPreferred(0x12);
  BLEDevice::startAdvertising();
/*
  BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
  pAdvertising->addServiceUUID(SERVICE_UUID);
  pAdvertising->setScanResponse(true);
  pAdvertising->setMinPreferred(0x06); // functions that help with iPhone connections issue
  pAdvertising->setMinPreferred(0x12);
  BLEDevice::startAdvertising();
*/
  Serial.println("Waiting a client connection to notify...");
```

```
}

void loop() {

  if (deviceConnected) {

    pTxCharacteristic->setValue(&txValue, 1);
    pTxCharacteristic->notify();
    txValue++;
    delay(10); // bluetooth stack will go into congestion, if too many packets are sent
  }

  // disconnecting
  if (!deviceConnected && oldDeviceConnected) {
    delay(500);                 // give the bluetooth stack the chance to get things ready
    pServer->startAdvertising(); // restart advertising
    Serial.println("start advertising");
    oldDeviceConnected = deviceConnected;
  }
  // connecting
  if (deviceConnected && !oldDeviceConnected) {
    // do stuff here on connecting
    oldDeviceConnected = deviceConnected;
  }
}
```